High level specifications for project on Agentic AI
CCAI 2025-26
Prof. Daniela Giordano

Goal:

Design, implement and evaluate an Agentic AI system (a "Copilot for domain-specific task")

Scenario:

Blogger Support

You are a blogger writing in the domain of your choice (e.g, your profession, your hobbies, your interests…). You aim at establishing your blog as a source of trusted information and knowledge.

You plan to write every (n) day(s) a post. The post should be an article (you set the max length) that can address one of the following:

- Upcoming events in your town/region related to your topics
- "How to" posts. These are posts in which you share with your readers a selected resource (e.g. an article) about how to do a certain thing. Example: your blog is about sailing and you want to write a post about how to do winch maintenance.
- A review of a new product/technology
- Interesting advances/news related to the domain

Your task is to prototype an Agentic system with LangGraph to support you in your activity.

The Agentic system should have (at least) the following capabilities:

- Verify the accuracy of the information gathered in Internet
- Selects among the resources found in Internet, based on some notion of quality and interestingness
- Drafts a post for you to review (Note: implementation of human-in the loop is required)
- Suggests you the topics of possible posts.
- Keep memory of previous posts and suggest topics that have not been addressed recently.
- The agents plans a sequence of posts and justifies its choices, and proceeds to prepare them.

  Note: writing the prepared posts to an external DB is not required

Knowledge Graph Requirements

The system must maintain an **editorial Knowledge Graph** representing the evolving knowledge of the blog.

The Knowledge Graph must include at least:

- previously generated posts
- topics covered
- sources used
- key claims extracted from the content
- relationships between topics

The Knowledge Graph must be **actively used**, not only stored.

In particular, it must be queried at least in the following stages:

1. **Topic suggestion phase**
    - to avoid repetition
    - to identify gaps in coverage
2. **Post drafting phase**
    - to ensure consistency with previously published content
    - to connect the new post with existing topics

The Knowledge Graph must be **incrementally updated** after each approved post.

K-RAG (Knowledge Graph–augmented RAG) Requirements

The system must implement a **Knowledge Graph–augmented Retrieval-Augmented Generation (K-RAG)** approach.

This includes:

- document retrieval from external or local sources (RAG)
- integration of Knowledge Graph information into the retrieval process
- use of both structured knowledge (KG) and unstructured documents (RAG)

In particular, the system must:

- use the Knowledge Graph to **expand or refine retrieval queries**
- use retrieved documents to **support claims in the generated post**
- explicitly include **citations or references** in the generated content

The final output must clearly demonstrate that:

- the content is grounded in retrieved documents
- the reasoning is informed by the Knowledge Graph

Reasoning and Tool Usage

The system must implement an **agentic reasoning process** (e.g., ReAct-style).

This requires:

- explicit reasoning steps (e.g., Thought → Action → Observation)
- dynamic selection of tools based on the task
- justification of tool usage in the reasoning trace

The system must include at least the following tools:

- a **Search tool** for retrieving external information
- a **RAG retrieval tool** for document-based knowledge
- a **Knowledge Graph tool** for querying and updating structured knowledge

The system must also include **at least two additional tools** designed and implemented by the team.

Planning Requirements

The system must include a **planning capability** that:

- generates a sequence of blog posts
- justifies the order and selection of topics
- ensures diversity and coverage of the domain

The planning process must:

- take into account previously generated content (via the Knowledge Graph)
- avoid redundancy
- demonstrate coherent editorial strategy

Model Context Protocol (MCP) / State Management

The system must explicitly manage its internal state (context), including:

- user input
- reasoning trace
- tool outputs
- Knowledge Graph summaries
- planning information

The state must be:

- updated at each step of the agent execution
- used as input for subsequent reasoning steps

Human-in-the-loop Requirement

The system must include a **human-in-the-loop mechanism** for reviewing generated content.

Before updating the Knowledge Graph, the system must:

- present the generated post to the user
- allow the user to:
    - approve
    - modify
    - reject and regenerate

The Knowledge Graph must be updated **only after user approval**.

Tooling Requirements (specific)

The system must:

- use at least three core tools:
    - Search tool
    - RAG retrieval tool
    - Knowledge Graph tool
- implement at least two additional tools designed by the team
- select tools dynamically through the reasoning process
- provide a clear justification for each tool invocation

Implementation Requirements

- The final system must be implemented as a **modular architecture** (not a single notebook).
- The provided notebook can be used for experimentation, but the final submission must include:
    - a structured codebase
    - separate modules for graph, tools, RAG, and Knowledge Graph

---

Evaluation Requirement

The system must be evaluated through:

- qualitative analysis of generated posts
- assessment of source quality and grounding
- identification of at least three failure cases
- in terms of observability parameters available in Langsmith.

Suggested resources

1) **Project: Ambient Agents with LangGraph (tutto)**

https://academy.langchain.com/courses/ambient-agents

2) **Project: Deep research with LangGraph (primi 3 notebook)**

https://academy.langchain.com/courses/deep-research-with-langgraph

3) **Build a custom RAG agent with LangGraph**

- self-rag (https://langchain-ai.github.io/langgraph/tutorials/rag/langgraph_self_rag/#setup)